BCSE307L COMPILER DESIGN · PROJECT A24 · TEAM 8 · REVIEW 3

Presenter notes: Ahmad Azim

Member 4: Testing, Evaluation and Release Owner · Reg. No. 24BCT0119
Your module: code emission (reduceA / reduceB), the simulators that verify every program, the driver, the test suite, the results and the repository.

1. Your part at a glance

4–5 min
your talk (slides 11, 12, 13b, 14, 18)

8–12 min
live demo: you drive the keyboard (slide 10)

27 / 27
checks pass: your headline number

2 backups
slides 19–20 for viva questions on your code

Slide	Title	Time	Your goal on this slide
10	End-to-end demo	with team	Run the commands; let each member explain their stage; finish with the evaluator's own input.
11	Test strategy	1.5 min	Two independent oracles; 27/27; what each test category proves.
12	Results	2 min	Correct (6,666/6,666), cheaper (−5.2% / −23.6%), linear time (O(n)).
13	Limitations (your half)	45 s	Virtual registers, no commutative rule, rejected operators, each with its remedy.
14	Repository	1.5 min	Three commands reproduce everything; results are committed and diffable.
18	References & acknowledgements	30 s	Papers, BURG as an unmodified tool, AI assistance acknowledged.
19–20	Backups	on demand	Show reduceA / simulateB / the test runner if the evaluator asks about your code.

2. Before the review

The night before

☐ `cd ~/Desktop/BURS/burs-instruction-selector && git pull`

☐ `make clean && make`: no warnings, ends with the `cc … -o build/burs` line

☐ `make test`: last line is `27 passed, 0 failed`

☐ Open the deck in PowerPoint; check slides 10–14 run in Presenter View

Fifteen minutes before

☐ Terminal in the repo root, font size ≥ 18 pt, window full height

☐ Run `make clean && make` once so the build is fresh

☐ Editor tabs open: `targetA.gr` (reduceA, line 58), `targetB.gr` (simulateB, line 134), `tests/run_tests.sh`, `tests/cases.txt`

- ☐ Charge the laptop; nothing in the demo needs internet
- ☐ `results/` folder and `docs/Review3_Document.pdf` open in Finder
- ☐ Close notifications and other apps

3. What to say, slide by slide

Slide 10 · End-to-end: one tree, two machines

You drive · all speak

SAY

"I'll run the complete system. Same expression, same IR tree, both targets." Run `./build/burs "a + b * c"`. "Target A loads three registers, multiplies, adds: five instructions, cost six, exactly the Review 2 result. Target B pushes a and b, then MULM folds c in as a memory operand, then ADD: four instructions, cost five. Both programs were executed by their simulators and both give 14, which is the IR's own value with a = 2, b = 3, c = 4."

SHOW / POINT AT

The **Retargeting summary** box at the bottom of the output, and the two **Simulation … OK** lines. Then hand over: Vineet explains the tree, Mayank the covers, Utsav the cost table (`-v`), you the code and the simulation.

Slide 11 · Test strategy: every output is checked twice

1.5 min

SAY

"We check every output with two independent oracles. First, regression: `run_tests.sh` runs 25 cases and compares the *complete* output, stdout, stderr and the exit status, byte for byte with a stored expected file. If a cover, a cost, an instruction or even an error message changes, the test fails. Second, semantics: every generated program is executed by the simulator for its ISA, and its result must equal the IR's own evaluation. The cases cover positive, boundary, rule-choice and negative inputs. On top of that, the 10-expression suite and an exhaustive check of every tree up to depth 2: 3,333 trees, 6,666 runs. All 27 checks pass."

SHOW / POINT AT

The 27/27 stat, then the six category cards. If time allows, run `make test` live; it takes under a second.

Slide 12 · Results: correct, cheaper and linear-time

2 min

SAY

"Three results. Correct: all 6,666 generated programs simulate to the right value. Cheaper: compared with a naive selector that emits one instruction per tree node under the same cost model, BURS is 5.2 percent cheaper on Target A and 23.6 percent cheaper on Target B, averaged over all 3,333 trees. B gains more because its memory and immediate operands absorb the right operand of most binary nodes. BURS can never be worse, because the naive cover is one of the covers it considers. Fast: labeling stays around 10 to 17 nanoseconds per node from 405 to almost 240,000 nodes. That is linear time, because burg precomputed the dynamic programming into tables, so each node is one lookup."

SHOW / POINT AT

Left chart: grey 7.26 → teal 6.88 → amber 5.55. Right chart: the two flat lines (the slight rise is cache effects on big trees, not the algorithm).

Slide 13 · Limitations (your half, after Utsav)

45 s

SAY

"On the implementation side: Target A uses virtual registers with no allocation, so the 20-variable deep test uses 39 registers; the fix is Sethi–Ullman or linear-scan allocation. There is no commutative rule, so `7 + a` costs 3 instead of 2. Our own test `rule_const_on_left` documents that on purpose, and the fix is one grammar rule. Unsupported operators such as division and unary minus are rejected with a clear error and exit status 2 instead of producing wrong code."

Slide 14 · Repository and reproducibility

1.5 min

SAY

"Everything is on GitHub, public. The only dependencies are a C compiler and make; the BURG generator is bundled and built by the Makefile. Three commands reproduce the project: clone, `make`, `make test`. `make results` regenerates every evidence file in `results/`. Those files are committed, so anyone can diff their run against ours. The layout follows the structure we proposed in Review 1."

SHOW / POINT AT

If asked, open the repository in the browser and the `results/` folder. `make` has three stages: build burg → `burg -p burmA` / `-p burmB` → compile and link.

Slide 18 · References and acknowledgements

30 s

SAY

"Our two core sources are Aho, Ganapathi and Tjiang on tree matching with dynamic programming, and Fraser, Hanson and Proebsting on BURG. The BURG generator in `third_party` is their tool in Chen Zhexuan's modernised version, used unmodified as a build tool under its own licence; it is not counted as our code. Implementation and documentation were assisted by an AI coding assistant, as recorded in the commits and the report; we reviewed and tested everything and are responsible for it."

4. Live demo runbook

Run from the repository root. `burs` below means `./build/burs`. Each step shows what appears and what to point at.

#	Command	What you will see	What to say
1	`make clean && make`	burg compiled, two `burg -p …` lines, final `cc` line; no warnings	"Three stages: build the generator, generate both matchers, link one program."
2	`burs "a + b * c"`	IR tree, two covers, two programs, both OK, summary A 5/6, B 4/5	The slide 10 script above.
3	`burs "[[a] + 1]"`	Both targets cost 4; value 2003 (mem[mem[2]+1])	"Boundary: a load of a load, with ADDI inside the address."
4	`burs "(a + b"`	`ERROR: … column 7: expected ')' but found end of input`, exit 2	"Invalid input is caught by the parser with the exact column, before selection."
5	`burs -v -t A "load(a) + 4"`	Cost table; `reg: ADD(reg,imm)` cost 3 at n4	Utsav explains; you point at the code and `Simulation 1006 OK`.
6	`make test`	27 PASS lines, then `27 passed, 0 failed`	"Byte-exact regression plus simulation on every case."
7	`burs --exhaustive`	3,333 trees, 0 failures, naive 7.26 vs 6.88 / 5.55	"Every small tree, both targets, and the comparison on slide 12."
8	`burs -i`	`burs>` prompt	"Please give us any expression." Type it, then `quit`.

Evaluator inputs: how to read any output in four steps

1. **IR tree**: check it matches the expression (`*` binds tighter; `a - b - c` is `(a - b) - c`).
2. **Cover**: one line per selected rule, top-down from the root, with node id and cost.
3. **Code**: emitted in postorder, children before parents, left to right.
4. **Simulation**: the program's result must equal **Reference**; `OK` means verified.

If something unexpected happens

Situation	Response
Evaluator types `a / b` or `-a`	"Division and unary minus are outside the approved IR, so the parser rejects them with exit status 2. Unary minus can be written `(0 - a)`." Run `burs "0 - a"` to show it.
Asks to change variable values	`burs -D a=10 -D b=7 "a * b - 3"`. The simulation uses the given values.
Asks for one target only	`-t A` or `-t B`.
Very long expression	Works; a 39-instruction case is in the test suite (`bnd_deep_nesting`). Limit: 512 instructions, then a clear error.
`build/burs` missing or stale	`make clean && make` (about 2 seconds).
A test fails after someone edited a file	`git status` and `git diff` to show what changed; `git checkout -- .` restores the committed version.

5. Know your code

Where	Function	What it does (say it like this)
`src/targets/targetA/targetA.gr:58`	`reduceA`	Called once per selected rule, after the children. `imm: CONST` emits nothing and returns `#k` for the parent to fold. Every other rule takes a fresh virtual register, emits one instruction and returns that register.
`targetA.gr:114`	`simulateA`	Executes the emitted text on a register file. Tracks which registers are defined, distinguishes `LOAD rd,[var]` from `LOAD rd,[rN]` (memory), and returns the value in the result register.
`src/targets/targetB/targetB.gr:69`	`reduceB`	`mem: VAR` and `imm: CONST` return operands `[a]` and `#k`. The chain rules `stk: mem` / `stk: imm` emit `PUSH` / `PUSHI`. Plain `ADD/SUB/MUL` use the stack; `…M` and `…I` forms take the right operand.
`targetB.gr:134`	`simulateB`	Runs a value stack: checks underflow, checks that `M` forms get a memory operand and `I` forms an immediate, and requires final depth exactly 1.
`src/burs/engine.c:138–171`	`burs_emit`, `burs_operand`, `burs_new_reg`, `burs_split`	Shared helpers: append an instruction (with an overflow guard at 512), build an operand string, allocate `rN`, and split an instruction line into opcode and operands for the simulators.

`src/main/main.c:110`	`run_target`	The pipeline for one target: label → (cost table with −v) → cover cost → print cover → reduce (emit) → simulate → compare with `ir_eval`.
`main.c:180, 227`	`process`, `run_suite`	Parse, print the IR, run every selected target, print the retargeting summary. The suite runs the 10 review expressions quietly and prints PASS/FAIL.
`main.c:275, 289`	`naive_cost`, `run_exhaustive`	Builds all 3,333 trees of depth ≤ 2 over {a, b, 3}; runs both targets; compares with the naive cost (one per node, MUL 2).
`main.c:361, 384`	`random_tree`, `run_bench`	Fixed-seed random balanced trees of depth 8–18; times `burs_label` + `burs_cost` with `clock()`, best of 5 runs.
`tests/run_tests.sh`	–	For each line of `cases.txt`: run, capture stdout + stderr + `exit=N`, `cmp` with `tests/expected/<id>.out`; then run `--suite` and `--exhaustive`. Non-zero exit on any failure. `--update` rewrites expected files after a deliberate change.

Emission in one picture: `a + b * c` on Target B

```
burs_reduce(ADD, stk)   rule stk: ADD(stk,stk)  → reduce children first
  burs_reduce(a, stk)   rule stk: mem   → child mem: VAR returns "[a]"  → PUSH [a]
  burs_reduce(MUL, stk) rule stk: MUL(stk,mem)
     burs_reduce(b, stk)  → PUSH [b]
     burs_reduce(c, mem)  → returns "[c]", no code
     → MULM [c]
  → ADD                     simulateB: 2, 3 → 3*4=12 → 2+12 = 14 = ir_eval  OK
```

Exit codes (asked often)

`0` success · `1` usage error (bad option, unknown target) · `2` invalid expression · `3` no cover, simulation mismatch or too many instructions.

6. Numbers cheat sheet

Fact	Value
Checks in `make test`	27 = 25 stored cases + `--suite` + `--exhaustive`; 27 passed, 0 failed
Stored cases by type	6 positive, 6 boundary, 5 rule choice, 8 negative
Exhaustive check	3,333 trees × 2 targets = 6,666 runs, 0 failures; max cover cost A 10, B 8
Average cover cost	naive 7.26 · Target A 6.88 (−5.2%, cheaper on 1,146 trees) · Target B 5.55 (−23.6%, cheaper on 3,267 trees)

Benchmark	405 → 239,710 nodes; A 9.9 → 14.7 ns/node; B 9.9 → 17.0 ns/node; 240k nodes in ≈ 4 ms
`a + b * c`	A: 5 instructions, cost 6 · B: 4 instructions, cost 5 · value 14
`load(a) + 4`	A: `LOAD, LOAD, ADDI` cost 3 (naive 4) · B: `PUSH, LOAD, ADDI` cost 3 · value 1006
Rules / nonterminals	A: 9 rules, reg + imm · B: 14 rules, stk + mem + imm
Cost model	1 per instruction; every multiply costs 2; operand-only rules cost 0
Test values	a = 2, b = 3, … z = 27 (other names: a hash, 2–51); memory: mem[x] = 1000 + x
Code size	≈ 1,900 lines of team code in `src/`, `tests/` and the Makefile, excluding the BURG tool

7. Likely viva questions on your module

Q How do you know the generated code is correct?

Every program is executed by a simulator for its ISA and the result is compared with `ir_eval` of the tree, which is an independent definition of what the expression means. On top of that, the regression suite compares the full output byte for byte, and the exhaustive check does this for all 3,333 trees of depth ≤ 2 on both targets. No failures.

Q Why simulate the text instead of trusting the emitter?

The simulator parses exactly the text a user sees, so it verifies the printed program, not an internal structure. It also checks things the emitter could get wrong: reading an undefined register, stack underflow, a memory form given an immediate, or a stack left with more than one value.

Q What does a byte-exact regression test catch that the simulator does not?

A program can be correct but different: a worse cover, a changed cost, different register numbering or a changed error message. Comparing the whole output catches any unintended change; the simulator only proves the value is right.

Q What is the naive baseline, and why is BURS never worse?

It emits one instruction per IR node (LOAD/LOADI or PUSH/PUSHI for leaves, the plain operator for interior nodes) under the same costs. That cover is always available in both grammars, and BURS returns the minimum-cost cover, so its cost is always ≤ the baseline, and strictly lower whenever an immediate or memory form applies.

Q Why does Target B gain so much more than Target A?

B has memory and immediate forms for ADD, SUB and MUL, so whenever the right operand is a variable or a constant, one `PUSH` disappears; that applies to 3,267 of 3,333 trees. A only has immediates for ADD and SUB on the right side, and no memory-operand arithmetic.

Q Why does time per node rise slightly on big trees if the algorithm is linear?

The work per node is constant (one table lookup), but larger trees no longer fit in the CPU cache, so each memory access is slower. A 600× bigger tree costs about 1.5–1.7× more per node, not 600×, which is what linear time looks like on real hardware.

Q How is the benchmark measured?

Random balanced trees with a fixed seed (so runs are repeatable), depths 8 to 18. The tree is built first; only `burs_label` plus the `burs_cost` backtrack are timed with `clock()`, and the best of five runs is reported. Timings vary by machine; the shape is what matters.

Q Why stop the exhaustive check at depth 2?

Depth 2 already exercises every rule of both grammars in every leaf combination (3,333 trees). Depth 3 would be $3{,}333 + 3 \times 3{,}333^2 \approx 33$ million trees: possible, but it adds run time without exercising new rules.

Q In what order are instructions emitted, and why does it work for a stack machine?

`burs_reduce` reduces a rule's children first, left to right, then calls the target's reduce: postorder. For a stack machine postorder is exactly evaluation order: `a − b` pushes a, then b, then `SUB` pops b and a and pushes a − b.

Q Why virtual registers? What happens on a real machine?

Register allocation was outside the approved scope; virtual registers keep the mapping from cover to code one-to-one. A real back end would run an allocator next, such as Sethi–Ullman numbering for trees or linear scan. It is listed as future work.

Q How would you add a new test?

Add a line `id | options | expression` to `tests/cases.txt`, run `sh tests/run_tests.sh --update`, read the new `tests/expected/<id>.out` to confirm it is right, and commit both. `--update` is only used after a deliberate change, and the diff is reviewed in git.

Q How would you add a third target?

Write `targetC.gr` with the same `%term` line, its rules and costs, a `reduce` and a `simulate`, and export `const Target targetC`. Add one `burg -p burmC` rule to the Makefile and `&targetC` to `all_targets[]` in `main.c`. The IR, parser, engine and tests do not change.

Q How do two burg-generated matchers link into one program?

`burg -p burmA` and `-p burmB` prefix every generated symbol (`burmA_label`, `burmB_rule`, …), so there are no name clashes. Each target's footer packs its functions into one `const Target`, and the driver keeps them in an array.

Q What happens with invalid input, or a huge expression?

The parser rejects malformed or unsupported input with the column number and exit status 2, before selection. Integer constants that would overflow are rejected too. Emission stops at 512 instructions with an error rather than overrunning the buffer.

Q What changed for your module since Review 2?

Target B's emitter and simulator (defect D3 closed); the automated regression suite (6 manual checks → 27 automated); the exhaustive check, baseline comparison and benchmark; the public repository and committed results. During final integration I also fixed D4 (`-t A` still ran both targets), D5 (a typo in a test case) and D6 (a misleading exhaustive summary).

Q Is the BURG code yours? Did you use AI tools?

Answer plainly. "BURG in `third_party` is the original authors' tool, used unmodified to generate the matchers; it's credited and not counted as our code. The implementation and documentation were developed with help from an AI coding assistant, which is acknowledged in the report and recorded in the commit history. I have reviewed, run and tested all of it and can explain any part."

If you do not know an answer: say what you do know, show the relevant output or file (`grep -n` finds any function in seconds), and offer to follow up. Do not guess numbers; they are all in section 6 and in `results/`.

Every number in these notes comes from `results/` in the repository (commit 4281e72 or later). Regenerate with `make results`.